

Code-Layout, Readability and Reusability

Date	30 JUNE 2026
Team ID	XXXXXX
Project Name	Credit Card Approval prediction
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Used consistent 4-space indentation in Python and proper indentation in HTML.
2	Proper File Structure	Files and folders are logically organized	Yes	Project organized into app.py, templates/, model (.pkl) files, scaler, encoders, dataset, and notebook.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Variables such as income, age, family_members, years_employed, and prediction are descriptive.
4	Function / Method Names	Functions are descriptively named	Yes	Functions such as home() and predict() clearly describe their purpose.
5	Code Comments	Inline and block comments explain logic	Yes	Added comments for data preprocessing, model loading, feature scaling, prediction, and Flask routes.
6	Modular Design	Code is split into reusable functions/modules	Partial	The project separates preprocessing, model training, and web application, but most logic is contained in a single Flask file.
7	No Redundant Code	Duplicate or unused code is removed	Yes	Unused features and duplicate preprocessing steps were removed before the final implementation.
8	Error Handling	Exceptions and errors are handled gracefully	Partial	Flask handles input submission, but advanced exception handling and custom error messages were not implemented.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	Data Preprocessing Module	Python, Pandas	Used for cleaning data, feature engineering, and preprocessing before model training	High

2	Label Encoding Module	Python, Scikit-learn (LabelEncoder)	Used for encoding categorical features during training and Flask prediction	High
3	Feature Scaling Module	Python, Scikit-learn (StandardScaler)	Used to scale features during model training and web application prediction	High
4	XGBoost Prediction Module	Python, XGBoost, Pickle	Used to load the trained model and predict credit card approval in the Flask application	High
5	Flask Web Application	Python, Flask, HTML, CSS	Used to collect user input, process requests, and display prediction results	Medium

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Project files are well organized with separate notebook, Flask application, templates, trained model, scaler, and encoder files.
Readability	5	Meaningful variable names, proper indentation, and clear code flow make the implementation easy to understand.
Reusability	4	Preprocessing, feature scaling, model loading, and prediction modules can be reused in similar machine learning projects.
Documentation / Comments	4	Appropriate comments are included to explain preprocessing, model training, Flask routes, and prediction logic.
Overall Score	4.5/5	The project follows good coding practices, has a modular structure, and is maintainable. Minor improvements can be made by adding more exception handling and detailed documentation.